

Rapport Final de Projet

Movies ELK Platform

1. Contexte du Projet

Le projet **Movies ELK Platform** s'inscrit dans le cadre d'une validation de compétences sur la stack Elastic. L'objectif principal est de construire de bout en bout une plateforme de données reproductible, conteneurisée et robuste, permettant d'ingérer, de nettoyer, d'indexer, d'analyser et de rechercher des données cinématographiques.

Le jeu de données imposé est *movies_dataset.csv*, contenant un extrait de 10 000 films avec un nombre limité de dimensions (8 colonnes). Le projet a été réalisé avec la contrainte forte de ne pas altérer la réalité du jeu de données (interdiction d'inventer des colonnes manquantes) et d'utiliser l'architecture Gitflow pour le contrôle de version.

2. Architecture de la Solution

L'architecture repose sur la suite **ELK (Elasticsearch, Logstash, Kibana)** en version 8.13.0, déployée localement via **Docker Compose**. L'ensemble est orchestré pour assurer une isolation des données et une reproductibilité totale.

Composants principaux :

Logstash : Chargé de l'extraction des données depuis le fichier CSV, de la transformation (nettoyage, typage, enrichissement) et du chargement dans Elasticsearch.

Elasticsearch : Le cœur de stockage et moteur de recherche hébergeant deux index : un index brut et un index propre avec mapping rigoureux.

Kibana : L'interface de data visualisation connectée à l'index *movies_clean*.

FastAPI (Search API) : Un micro-service Python communiquant avec Elasticsearch pour exposer un moteur de recherche complet.

3. Données et Nettoyage

3.1. Le Dataset d'origine (*movies_dataset.csv*)

Le fichier source contient 10 000 lignes et 8 colonnes : *index*, *title*, *original_language*, *release_date*, *popularity*, *vote_average*, *vote_count*, et *overview* (synopsis textuel).

3.2. Anomalies Observées

1. **Structurelle** : La toute première ligne du CSV est une ligne vide, suivie de la ligne d'en-tête.
2. **Formats de Date** : Le format *dd-MM-yyyy* n'est pas nativement optimisé pour le tri chronologique.
3. **Données Manquantes** : ~95 films sans synopsis, 18 films sans date de sortie.
4. **Anomalies Métier** : Films avec notes mais peu de votes (données non significatives), dates dans le futur.

4. Règles de Nettoyage (Pipelines Logstash)

R1 - Filtrage initial : Les lignes dont l'*index* est vide ou vaut « index » sont exclues.

R2 - Normalisation textuelle : Suppression des espaces en début/fin et conversion en minuscules.

R3 - Typage fort : Conversion explicite en float, integer selon les champs.

R4 - Formatage des dates : Conversion *dd-MM-yyyy* en ISO 8601.

R5 - Enrichissement par tags de Qualité : Injection de marqueurs (*missing_overview*, *missing_release_date*, *low_vote_count*, *future_release*).

R6 - Booléens analytiques : Création de champs pré-calculés (*has_overview*, *has_release_date*, *is_rating_reliable*).

5. Impact Avant / Après

Le découpage en deux index (*movies_raw* vs *movies_clean*) permet de mesurer précisément l'impact du pipeline :

Métrique	movies_raw (Index brut)	movies_clean (Index nettoyé)	Bénéfice
Volume de documents	10 006	10 000	Disparition des scories
Typage	Tout en chaîne	Mixte (Text, Keyword, Date, Float)	Requêtes mathématiques
Gestion des dates	Chaîne ("10-06-2004")	Date (2004-06-10T00...)	Filtres temporels natifs
Qualité et fiabilité	Aucune visibilité	Tableau <i>quality_tags</i>	Filtrage par clic

6. Mapping Elasticsearch (*movies_clean*)

Le mapping de *movies_clean* a été défini de manière stricte (explicit mapping) pour optimiser la recherche et l'occupation disque.

Caractéristiques principales :

Analyzer Personnalisé : Un *movie_text_analyzer* associant tokenizer standard avec filtres *lowercase* et *asciifolding*.

Champs Textuels : *title* en type text et keyword, *overview* uniquement en text.

Champs Catégoriels : *original_language* et *quality_tags* typés en keyword.

7. Requêtes et Analyses

Une série de 12 requêtes métier a été développée et documentée pour démontrer la capacité de la plateforme :

1. **Recherches de pertinence** (*match*, *match_phrase*) : Retrouver des films spécifiques.
2. **Requêtes de qualité** (*term*, *range*) : Détection des anomalies.
3. **Agrégations** (*terms*, *histogram*, *stats*) : Répartition linguistique, analyse par décennie.
4. **Requêtes Booléennes Multi-critères** : Ciblage précis avec *must*, *should*, *must_not*.

8. Dashboard Kibana

Un tableau de bord complet offre une vue macro et micro du catalogue avec plusieurs types de visualisations :

1. **Métriques globales (KPIs)** : Nombre total de films, note moyenne globale.
2. **Répartition linguistique** (Pie Chart) : Proportion écrasante de l'anglais.
3. **Volume de production** (Bar Chart/Histogram) : Nombre de sorties par année.
4. **Corrélation Notes/Popularité** : Vérification de tendances.
5. **Indicateurs de Qualité** : Fréquence des `quality_tags`.
6. **Data Table** : Tableau détaillé permettant recherche et filtrage direct.

9. API de Recherche (Search API)

Un micro-service indépendant a été développé avec **FastAPI**. Ce service expose une route `/search` qui :

- Combine la puissance du `multi_match` d'Elasticsearch.
- Applique des filtres métier (langue, plages d'années, score minimum).
- Gère la pagination de manière native.
- Formate et tronque la réponse pour le front-end (synopsis limités à 150 caractères).

10. Gestion de Projet (Gitflow & Agile)

Le projet a été mené selon une méthodologie professionnelle :

- Gitflow** : Découpage stricte avec branches *main* (production), *dev* (intégration), et *feature/**.
- Planning Poker** : Ensemble des tâches estimé en story points.
- Documentation continue** : Chaque fonctionnalité accompagnée de sa documentation.

11. Limites et Améliorations Futures

Limites identifiées :

- Pauvreté du Dataset** : Absence de dimensions analytiques majeures (genres, budget, revenus).
- Architecture Single-Node** : Le cluster Elasticsearch sans réplicas n'est pas « production-ready ».

Améliorations envisagées :

1. **Enrichissement de données** : Brancher le pipeline sur l'API TMDb pour rapatrier genres et budgets.
2. **Sécurité (X-Pack)** : Activer l'authentification native d'Elasticsearch.
3. **Tests Automatisés** : Ajouter une suite pytest pour la Search API.